



# Resultado de Aprendizaje 1 (RA1)

Arquitecturas y Tecnologías de Programación sobre Clientes Web

Módulo: Desarrollo Web en Entorno Cliente (DAW)

# Índice de la Presentación

---

## Fundamentos y Arquitectura

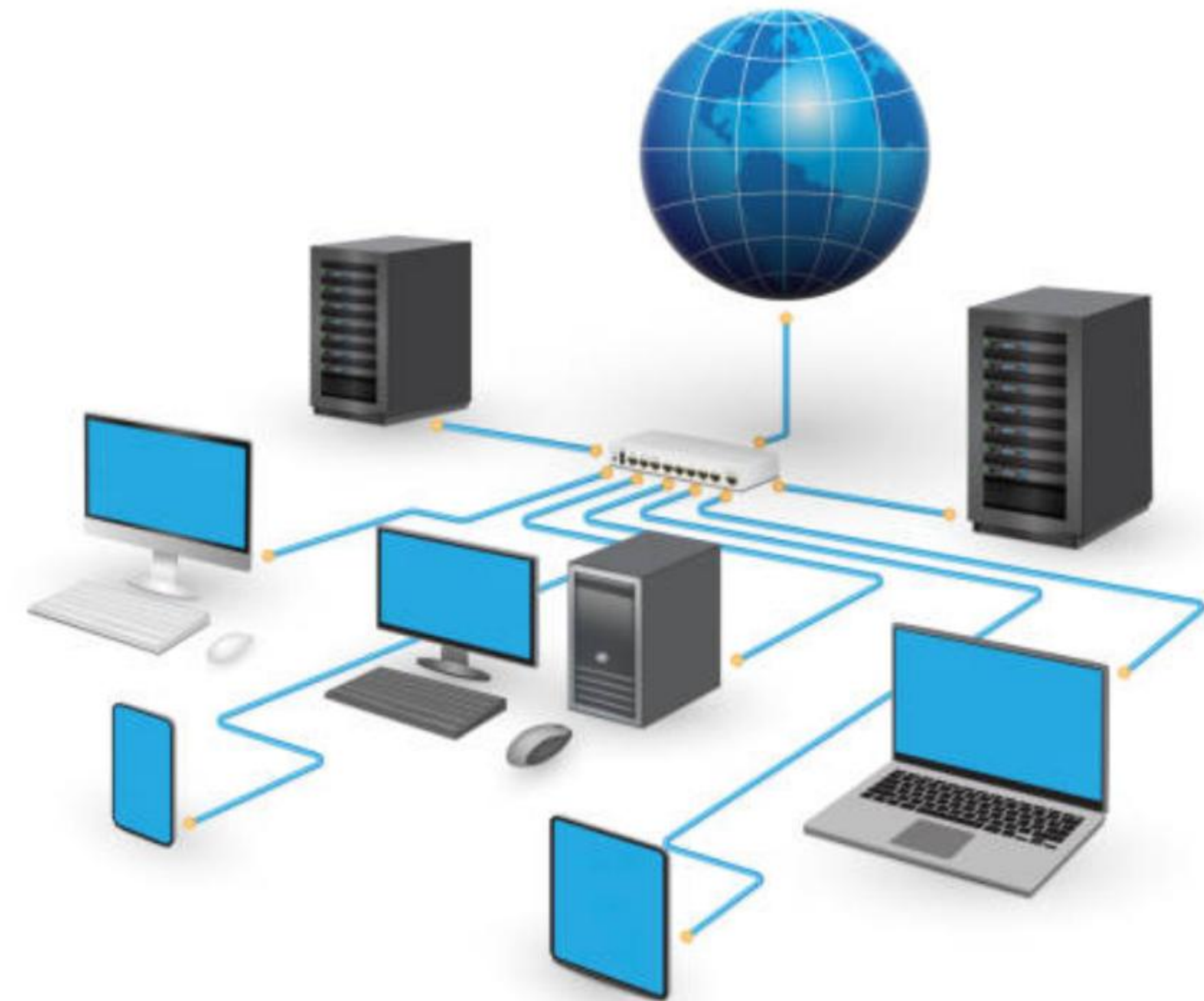
- **Parte 1:** Modelo Cliente/Servidor y Distribución del Procesamiento.
- **Parte 2:** Evolución Arquitectónica Web: De MPA a SPA e Híbridas.
- **Parte 3:** Anatomía y Capas Internas de los Navegadores Web.
- **Parte 4:** Motores JavaScript y Compilación JIT.
- **Parte 5:** Concurrencia Monohilo, Call Stack y Event Loop.

## Ejecución, Lenguajes y Seguridad

- **Parte 6:** El Pipeline de Renderizado Crítico.
- **Parte 7:** Ecosistema: JS, TypeScript y WebAssembly.
- **Parte 8:** Scripts vs Programación Tradicional y Sandbox.
- **Parte 9:** Seguridad Web: SOP y CORS.
- **Parte 10:** Integración HTML5, DevTools y Lighthouse.

# 1. Modelo Cliente/Servidor

- **Flujo de Comunicación:** Protocolo HTTP/HTTPS mediante ciclos de Petición (Request) y Respuesta (Response).
- **Servidor (Backend):** Responsable del acceso a BD, custodia de secretos y validación definitiva. Asegura integridad, pero añade latencia (RTT).
- **Cliente (Frontend):** Renderizado UI e interactividad. Aprovecha la CPU/RAM del usuario (cómputo descentralizado).
- **Validación Local:** Mejora enormemente la experiencia de usuario (UX) dando respuesta inmediata sin saturar la red.
- **Regla de oro:** El cliente es un **entorno no confiable**. Las validaciones frontend no eximen de validar en backend.



## 2. Evolución de Arquitecturas Web

| Característica     | MPA Clásica (Tradicional)  | SPA (Single-Page App)      | SSR Híbrido Moderno        |
|--------------------|----------------------------|----------------------------|----------------------------|
| Tecnologías        | PHP, JSP, ASP.NET          | React, Angular, Vue        | Next.js, Astro, Nuxt       |
| Generación Inicial | Servidor (HTML completo)   | Cliente (HTML cascarón)    | Servidor (HTML completo)   |
| Navegación         | Recarga completa de página | Interceptada por JS        | Interceptada tras hidratar |
| Tráfico de Red     | Descarga de HTML+Recursos  | Peticiones JSON mínimas    | JSON / Server Components   |
| Indexación (SEO)   | Excelente de forma nativa  | Requiere renderizado extra | Excelente y optimizada     |

### 3. Anatomía Interna de los Navegadores

---



#### User Interface (UI)

Comprende la barra de direcciones, pestañas, marcadores y controles. El **Browser Engine** orquesta las acciones entre la UI y los motores subyacentes, actuando como puente principal.



#### Motor de Renderizado

Procesa HTML y CSS para calcular geometría y dibujar en pantalla. Ejemplos líderes: **Blink** (Chrome, Edge), **Gecko** (Firefox) y **WebKit** (Safari / iOS).



#### Motor JavaScript

Interpreta y compila código ECMAScript a instrucciones de CPU. No debe confundirse con el motor de render. Destacan: **V8** (Google), **SpiderMonkey** (Mozilla) y **Nitro** (Apple).

## 4. Compilación JIT en el Motor V8

---



### 1. Scanner & Parser

El código fuente (texto) es descompuesto en tokens sintácticos para generar el **AST (Abstract Syntax Tree)**, una estructura de datos que representa lógicamente el programa.



### 2. Ignition (Intérprete)

Genera rápidamente **Bytecode** a partir del AST. El código se ejecuta inmediatamente sin esperas, proporcionando tiempos de inicio muy rápidos en la web.



### 3. TurboFan (JIT)

El Profiler detecta funciones muy usadas (*Hot Functions*). TurboFan compila el bytecode a **Código Máquina Nativo** super optimizado. Puede sufrir deoptimización si cambian los tipos de datos.

## 5. Event Loop y Concurrencia Monohilo

---

- **La Paradoja:** JS es de un hilo (Call Stack), pero el navegador proporciona APIs asíncronas en hilos de fondo para no congelar la UI.
- **Call Stack:** Ejecuta funciones síncronas una a una (LIFO).
- **Microtask Queue:** Alta prioridad. Aloja Callbacks de Promesas y MutationObservers.
- **Task Queue (Macrotareas):** Funciones como setTimeout, eventos de ratón o I/O.
- **El Algoritmo:** El Event Loop espera a que el Call Stack esté vacío, luego vacía *todas* las Microtareas, renderiza la pantalla y finalmente procesa *una* Macrotarea.



## 6. Pipeline de Renderizado Crítico

---

✓ 1. **Árbol DOM:** Nodos lógicos generados a partir de las etiquetas del documento HTML.

✓ 2. **CSSOM:** Árbol de reglas y estilos calculados desde las hojas de estilo CSS.

✓ 3. **Render Tree:** Fusión de DOM y CSSOM. Descarta elementos no visibles (`display: none`).

✓ 4. **Layout / Reflow:** Cálculo matemático exacto de dimensiones y coordenadas de cada caja en pantalla.

✓ 5. **Paint:** Rasterizado de píxeles (texto, colores, sombras) en capas de mapa de bits.

✓ 6. **Composite:** La tarjeta gráfica (GPU) unifica todas las capas y las proyecta al monitor.

⚠ ¡Cuidado con el Layout Thrashing! Alternar lecturas y escrituras del DOM en bucles fuerza recalculer el Layout repetidamente, hundiendo los FPS de la animación.

## 7. Ecosistema de Lenguajes Cliente

---



### JavaScript (ES6+)

El estándar nativo regulado por el comité TC39. Gestiona el DOM, interactividad y peticiones de red. Su evolución anual permite sintaxis moderna como ``async/await``.



### TypeScript

Superset empresarial creado por Microsoft que aporta tipos estáticos y validación. No corre en el navegador: se **transpila** a JavaScript estándar asegurando código libre de bugs tipográficos.



### WebAssembly (Wasm)

Formato binario para la web. Se compila desde C++, Rust o Go. Proporciona aceleración casi nativa para tareas pesadas (motores 3D, edición de vídeo) complementando a JS.

## 8. Scripts Web vs. Programación Tradicional

---

### Diferencias Estructurales

- **Compilación:** Softwares tradicionales compilan previo a la ejecución (AOT). La Web compila al vuelo en memoria (JIT).
- **Gestión de Memoria:** En C/C++ es manual (punteros). JS es dinámico y usa un Recolector de Basura (Garbage Collector).
- **Tipado:** Históricamente dinámico y de coerción débil en scripts, frente al estricto tipado tradicional.

### El Modelo Sandbox de Seguridad

- **Aislamiento:** Al ejecutar código de internet no confiable, el navegador lo confina en una "caja de arena".
- **Restricciones de E/S:** Prohibido leer el disco duro del usuario (salvo API de Permisos o inputs).
- **Site Isolation:** Cada pestaña corre en procesos separados del SO mitigando vulnerabilidades hardware como Spectre.

## 9. Seguridad de Red: SOP y CORS

---

- **Same-Origin Policy (SOP):** Política crucial. Un script solo accede a recursos de otro dominio si comparten *exactamente* el **Protocolo, Host y Puerto**.
- **CORS (Cross-Origin Resource Sharing):** Mecanismo que flexibiliza el SOP permitiendo peticiones seguras entre dominios distintos mediante cabeceras HTTP explícitas del servidor (`Access-Control-Allow-Origin`).
- **Petición Preflight:** Para métodos que alteran datos (PUT, DELETE), el navegador envía preventivamente una solicitud **OPTIONS** para verificar la autorización del servidor.



# 10. Herramientas, Módulos y Auditoría

---



## Integración HTML5

Atributos vitales: `async` (ejecución sin orden) y **`defer`** (respetar el orden y el parseo). `type="module"` aporta modularidad nativa (ES6), aislamiento y strict mode automático.



## DevTools Integradas

El panel F12 es indispensable: **Elements** (DOM/CSS), **Console** (REPL), **Sources** (Depurador con Breakpoints), y **Network** para el análisis del tráfico HTTP.



## Auditoría Web Vitals

Google Lighthouse mide las métricas vitales: **LCP** (velocidad visual principal), **INP** (latencia de interactividad) y **CLS** (estabilidad visual para evitar saltos en pantalla).

# Resumen Ejecutivo de Competencias del RA1

| Tema Principal          | Concepto Clave de Evaluación y Retención                                      |
|-------------------------|-------------------------------------------------------------------------------|
| Modelo Cliente/Servidor | Validación UX (rápida) en cliente vs seguridad e integridad real en servidor. |
| Arquitectura de Apps    | SPAs con History API y evolución a hidratación en frameworks SSR modernos.    |
| Motores y Compilación   | Renderizado (Blink, WebKit) separado del motor JS (V8 con JIT TurboFan).      |
| Ejecución y Flujos      | Call Stack monohilo, Event Loop y prioridad de Promesas (Microtask Queue).    |
| Critical Rendering Path | Fases gráficas y peligro de reflows masivos por Layout Thrashing con JS.      |
| Ecosistema y Seguridad  | Tipado TypeScript (transpilado), Sandbox aislante, y validaciones SOP/CORS.   |

# Image Sources

---



<https://media.istockphoto.com/id/2149328154/vector/lan-network-3d-diagram.jpg?s=612x612&w=0&k=20&c=0sWs7pIVl8iAwXR1eNJVwWEROHpmzkNcM3BuGg4nQOg=>

Source: [www.istockphoto.com](https://www.istockphoto.com)

---



<https://as-api-ws-cdn-1.tdacestudio.com/web/content/1675e24656194febcb49a5f8c8e9d71e.jpeg>

Source: [acestudio.ai](https://www.tdacestudio.ai)

---



[https://media.easy-peasy.ai/66907430-4c24-408b-8bf0-7f9210e4a5f7/02360a1a-3eea-4385-b886-03fb43af53d4\\_medium.webp](https://media.easy-peasy.ai/66907430-4c24-408b-8bf0-7f9210e4a5f7/02360a1a-3eea-4385-b886-03fb43af53d4_medium.webp)

Source: [easy-peasy.ai](https://www.easy-peasy.ai)